



CONCORRÊNCIA E MULTITHREADING

AULA 01 - OVERVIEW



AGENDA

- ❑ Importance of concurrency in modern applications
- ❑ Basic concepts of concurrency
- ❑ Evolution of concurrency in Java
- ❑ Modern Java Concurrency
- ❑ Best Practices and Design Patterns
- ❑ Real-world Applications and Use Cases
- ❑ Conclusion and Future Trends



GENERAL OVERVIEW

IMPORTANCE OF CONCURRENCY IN MODERN APPLICATIONS

Concurrency isn't just a nice-to-have feature. It's a necessity.

- ❑ Performance: Concurrent programming allows applications to utilize multi-core processors effectively, significantly improving performance and responsiveness.
- ❑ Resource Utilization: Concurrency enables efficient use of system resources, allowing applications to perform multiple tasks simultaneously without wasting CPU cycles.
- ❑ Scalability: As user bases grow and data volumes increase, concurrent applications can scale more effectively to handle increased load.
- ❑ User Experience: Concurrency allows applications to remain responsive while performing time-consuming tasks in the background, enhancing user experience.
- ❑ Real-time Systems: Many modern applications, from financial trading platforms to online gaming, require real-time processing that's only possible with efficient concurrency.
- ❑ Big Data and AI: Processing large datasets and running complex algorithms often require parallel execution to be feasible.



BASIC CONCEPTS OF CONCURRENCY

Threads and processes

- ❑ Process: A process is an independent program running in its own memory space. Each process has its own resources, including memory, file handles, and security context.
- ❑ Thread: A thread is a lightweight unit of execution within a process. Multiple threads in a process share the same memory space and resources.
- ❑ Key points:
 - ❑ Threads allow for concurrent execution within a single process.
 - ❑ They share memory, making communication easier but also introducing potential synchronization issues.
 - ❑ The JVM and operating system manage thread scheduling.



BASIC CONCEPTS OF CONCURRENCY

Synchronization and thread safety

- ❑ Synchronization: Coordinating the execution of threads to ensure they don't interfere with each other when accessing shared resources.
- ❑ Thread safety: A class or method is thread-safe if it behaves correctly when accessed from multiple threads simultaneously.
- ❑ Synchronization mechanisms:
 - ❑ The synchronized keyword
 - ❑ Lock objects (ReentrantLock, ReadWriteLock)
 - ❑ Atomic classes (AtomicInteger, AtomicReference, etc.)
 - ❑ Volatile keyword for ensuring visibility of changes across threads



EVOLUTION OF CONCURRENCY IN JAVA

From Java 1.0 to Java 21

- ❑ Java 1.0-1.4: Basic thread support with Thread class and synchronized keyword.
- ❑ Java 5: Introduction of the `java.util.concurrent` package.
- ❑ Java 7: Fork/Join framework for parallel processing.
- ❑ Java 8: `CompletableFuture` and parallel streams.
- ❑ Java 9+: Continued refinements and additions to concurrent APIs.
- ❑ Java 21: Virtual threads



EVOLUTION OF CONCURRENCY IN JAVA

Key milestones and improvements

- ❑ Java 5 (2004): This was a game-changer for concurrency in Java.
 - ❑ Executor framework for managing thread pools
 - ❑ Concurrent collections (ConcurrentHashMap, CopyOnWriteArrayList, etc.)
 - ❑ Atomic variables for lock-free thread-safe programming
 - ❑ New synchronizers like CountdownLatch and CyclicBarrier
- ❑ Java 7 (2011):
 - ❑ Fork/Join framework for divide-and-conquer parallel processing
 - ❑ TransferQueue for producer-consumer scenarios
- ❑ Java 8 (2014):
 - ❑ CompletableFuture for composing asynchronous operations
 - ❑ Parallel streams for easy parallelization of collection processing



EVOLUTION OF CONCURRENCY IN JAVA

Key milestones and improvements

- ❑ Java 9 (2017):
 - ❑ Reactive Streams API for asynchronous stream processing with non-blocking back pressure
- ❑ Java 21 (2022):
 - ❑ Virtual Threads - lightweight threads for high-throughput concurrent applications
- ❑ Java 24 (2025):
 - ❑ Structured Concurrency (Fifth Preview) - Simplify concurrent programming by introducing an API for structured concurrency. Treats groups of related tasks running in different threads as single units of work, thereby streamlining error handling and cancellation



MODERN JAVA CONCURRENCY

EXECUTORS AND THREAD POOLS

The Executor framework separates task submission from task execution, providing a more flexible and efficient way to manage threads

- ❑ Key points:
 - ❑ ExecutorService interface for managing thread lifecycle
 - ❑ Different types of thread pools:
 - ❑ Fixed thread pool
 - ❑ Cached thread pool
 - ❑ Scheduled thread pool
 - ❑ Single-threaded executor
- ❑ Benefits:
 - ❑ Reuse of threads reduces overhead
 - ❑ Improved application performance and resource management



EXECUTORS AND THREAD POOLS

The Executor framework separates task submission from task execution, providing a more flexible and efficient way to manage threads

Example:

```
ExecutorService executor = Executors.newFixedThreadPool(4);  
executor.submit(() -> System.out.println("Task executed"));
```



CONCURRENT COLLECTIONS

Thread-safe collections designed for high concurrency scenarios

- ❑ Key classes:
 - ❑ ConcurrentHashMap: A highly concurrent, scalable alternative to `Collections.synchronizedMap()`
 - ❑ CopyOnWriteArrayList: Thread-safe variant of `ArrayList`, optimized for read-heavy scenarios
 - ❑ ConcurrentLinkedQueue: Non-blocking queue implementation
- ❑ Benefits:
 - ❑ Better performance in multi-threaded environments
 - ❑ Reduced need for external synchronization



CONCURRENT COLLECTIONS

Thread-safe collections designed for high concurrency scenarios

Example:

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();  
map.put("key", 1);  
map.computeIfPresent("key", (k, v) -> v + 1);
```



SYNCHRONIZERS

Utility classes that help coordinate the actions of multiple threads

- ❑ Key classes:
 - ❑ CountDownLatch: Allows one or more threads to wait until a set of operations in other threads completes
 - ❑ CyclicBarrier: Enables a set of threads to wait for each other to reach a common barrier point
 - ❑ Semaphore: Controls access to a shared resource through a counter



SYNCHRONIZERS

Utility classes that help coordinate the actions of multiple threads

Example:

```
CountDownLatch latch = new CountDownLatch(3);  
// ... start 3 threads that each call latch.countDown()  
latch.await(); // Wait for all 3 threads to finish
```



FORK/JOIN FRAMEWORK

A framework that helps you take advantage of multiple processors. It is designed for work that can be broken into smaller pieces recursively. The goal is to use all the available processing power to enhance the performance of your application.

- ❑ Key points:
 - ❑ It's an implementation of the `ExecutorService` interface
 - ❑ Designed for work-stealing algorithm
 - ❑ `ForkJoinPool` manages worker threads
 - ❑ `RecursiveTask` and `RecursiveAction` for creating tasks
- ❑ Benefits:
 - ❑ Efficient for problems that can be broken down into smaller subtasks
 - ❑ Automatic work balancing among threads



FORK/JOIN FRAMEWORK

A framework for parallel execution of recursively divisible tasks

Example:

```
class SumTask extends RecursiveTask<Long> {  
    // ... implementation  
}  
  
ForkJoinPool pool = new ForkJoinPool();  
Long result = pool.invoke(new SumTask(array, 0, array.length));
```



COMPLETABLEFUTURE

Provides a way to write asynchronous,
non-blocking code

- ❑ Key features:
 - ❑ Chaining of asynchronous operations
 - ❑ Combining multiple asynchronous computations
 - ❑ Exception handling for asynchronous tasks
- ❑ Benefits:
 - ❑ Simplified asynchronous programming model
 - ❑ Easy composition of multiple asynchronous operations



COMPLETABLEFUTURE

Provides a way to write asynchronous,
non-blocking code

Example:

```
CompletableFuture<String> future = CompletableFuture.supplyAsync() -> "Hello")  
    .thenApply(s -> s + " World")  
    .thenApply(String::toUpperCase);
```



PARALLEL STREAMS

Enables easy parallelization of stream operations

- ❑ Key points:
 - ❑ `parallel()` method to create a parallel stream
 - ❑ Leverages the Fork/Join framework under the hood
 - ❑ Suitable for large datasets and computationally intensive operations
- ❑ Benefits:
 - ❑ Easy way to parallelize operations on collections
 - ❑ Can significantly improve performance for suitable tasks



PARALLEL STREAMS

Enables easy parallelization of stream operations

Example:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);  
int sum = numbers.parallelStream()  
    .mapToInt(i -> i * 2)  
    .sum();
```



VIRTUAL THREADS

Feature that introduces lightweight, user-mode threads. Part of the Project Loom.

- ❑ Key points:
 - ❑ Managed by the JVM, not the OS
 - ❑ Enables a high number of concurrent tasks with low overhead
 - ❑ Designed to simplify concurrent programming
- ❑ Benefits:
 - ❑ Greatly increased application throughput for I/O-bound applications
 - ❑ Simplified programming model for concurrent applications
 - ❑ Reduced need for complex thread pooling strategies



VIRTUAL THREADS

Feature that introduces lightweight, user-mode threads. Part of the Project Loom.

- ❑ Key differences from platform threads:
 - ❑ Much lower memory footprint (about 1KB vs 1MB for platform threads)
 - ❑ Can support millions of virtual threads on a single JVM
 - ❑ Automatic yield during blocking operations, improving overall efficiency
- ❑ Considerations:
 - ❑ May require rethinking some existing concurrency patterns
 - ❑ Not a silver bullet



VIRTUAL THREADS

Feature that introduces lightweight, user-mode threads. Part of the Project Loom.

Example:

```
Thread.startVirtualThread() -> {  
    System.out.println("I'm a virtual thread!");  
});
```

Or using the new structured concurrency API:

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {  
    Subtask<String> subtask1 = scope.fork() -> fetchDataFromApi1();  
    Subtask<String> subtask2 = scope.fork() -> fetchDataFromApi2();  
  
    scope.join();  
    scope.throwIfFailed();  
  
    String result1 = subtask1.get();  
    String result2 = subtask2.get();  
}
```



BEST PRACTICES AND DESIGN PATTERNS

THREAD-SAFE PROGRAMMING TECHNIQUES

Thread-safe code is crucial for building robust concurrent applications

- ❑ Immutable objects: Immutable objects are inherently thread-safe, as they cannot be modified after creation.
- ❑ Encapsulation: Carefully encapsulate shared state and provide synchronized access to it.
- ❑ Defensive copying: Make defensive copies of mutable objects when passing them between threads.
- ❑ Atomic operations: Use atomic classes (AtomicInteger, AtomicReference, etc.) for fine-grained synchronization.
- ❑ ThreadLocal: Use ThreadLocal to create thread-confined variables.



AVOIDING COMMON PITFALLS

While concurrency brings many benefits, it also introduces potential issues that must be carefully managed

- ❑ Deadlocks:
 - ❑ Occur when two or more threads are blocked, each waiting for a resource held by another thread.
 - ❑ Prevent by following resource acquisition ordering, using timeouts, and avoiding nested locks.
- ❑ Race conditions:
 - ❑ Happen when the correctness of a computation depends on the relative timing or interleaving of multiple threads.
 - ❑ Mitigate by using proper synchronization, atomic operations, and avoiding shared mutable state.



PERFORMANCE CONSIDERATIONS

When working with concurrency, it's important to consider the performance implications of your design choices

- ❑ Thread pool sizing: Determine the optimal number of threads for your application's workload.
- ❑ Granularity of tasks: Balance the overhead of thread creation/management with the benefits of parallelism.
- ❑ Contention management: Minimize contention on shared resources to avoid throttling performance.
- ❑ Asynchronous processing: Use asynchronous APIs (e.g., `CompletableFuture`) to avoid blocking the main thread.
- ❑ Profiling and monitoring: Continuously monitor your application's concurrency behavior and identify bottlenecks.



REAL-WORLD APPLICATIONS AND USE CASES

CONCURRENT APPLICATIONS IN VARIOUS DOMAINS

Concurrency is a fundamental aspect of modern software development, and it is applied in a wide range of real-world applications

- ❑ Web servers and microservices:
 - ❑ Highly concurrent, asynchronous, and I/O-bound applications
 - ❑ Utilize features like `Executor`, `CompletableFuture`, and virtual threads
- ❑ Financial trading platforms:
 - ❑ Require low-latency, high-throughput processing of market data
 - ❑ Leverage concurrent data structures and lock-free algorithms
- ❑ Big data and stream processing:
 - ❑ Utilize parallel processing of large datasets using frameworks like Apache Spark
 - ❑ Leverage parallel streams



CONCURRENT APPLICATIONS IN VARIOUS DOMAINS

Concurrency is a fundamental aspect of modern software development, and it is applied in a wide range of real-world applications

- ❑ Games and simulations:
 - ❑ Require concurrent simulation of multiple entities, physics, and rendering
 - ❑ Use thread pools and task-based parallelism for efficient utilization of resources
- ❑ Mobile and embedded systems:
 - ❑ Concurrency is essential for responsive user interfaces and efficient resource management
 - ❑ Employ event-driven, asynchronous programming models



PERFORMANCE IMPROVEMENTS ACHIEVED THROUGH CONCURRENCY

Concurrency can lead to significant performance improvements in various scenarios

- ❑ Web server performance:
 - ❑ Switching from a traditional thread-per-request model to an asynchronous, event-driven architecture (e.g., using virtual threads) can increase throughput by an order of magnitude.
 - ❑ Leveraging `CompletableFuture` and non-blocking I/O can reduce latency and improve overall responsiveness.
- ❑ Data processing pipelines:
 - ❑ Parallelizing data transformation and aggregation tasks using parallel streams can provide significant speedups, especially for CPU-bound workloads.
 - ❑ Utilizing the Fork/Join framework for divide-and-conquer algorithms can lead to efficient parallel processing of large datasets.



PERFORMANCE IMPROVEMENTS ACHIEVED THROUGH CONCURRENCY

Concurrency can lead to significant performance improvements in various scenarios

- ❑ Reactive microservices:
 - ❑ Adopting a reactive programming model, combined with asynchronous communication and non-blocking I/O, can improve scalability and resilience of microservices.
 - ❑ Technologies like Mutiny, integrated with Java's concurrency APIs, enable the development of highly responsive, event-driven microservices.
- ❑ Real-time analytics and monitoring:
 - ❑ Concurrent processing of high-velocity data streams, using tools like Apache Kafka and Flink, allows for real-time analytics and anomaly detection.
 - ❑ Leveraging Java's concurrent collections and synchronization primitives can ensure efficient and thread-safe handling of data in these high-throughput scenarios.



WHAT'S NEXT

EMERGING TRENDS IN JAVA CONCURRENCY

- ❑ Continued evolution of Virtual Threads (Project Loom):
 - ❑ Expect further refinements and stabilization as the feature matures.
 - ❑ Potential for broader adoption and integration with other Java concurrency tools.
 - ❑ Final version of Structured Concurrency
- ❑ Improving performance and scalability:
 - ❑ Ongoing research and optimization of the JVM's concurrency internals.
 - ❑ Exploiting hardware-level concurrency features, such as newer processor architectures.



EMERGING TRENDS IN JAVA CONCURRENCY

- ❑ Tooling and ecosystem advancements:
 - ❑ Improved profiling and monitoring tools for concurrent applications.
 - ❑ Integration of concurrency best practices into development frameworks and libraries.



ATÉ A PRÓXIMA!